

Dispensa · Esercitazione di preparazione al compitino

Esercizi:

1. verificare se una sequenza di numeri è crescente;
 2. stampare un rettangolo di asterischi di dimensione $n \times m$;
 3. stampare una scacchiera di dimensione $n \times m$;
 4. costruire la lista degli elementi comuni tra due liste;
 5. invertire il contenuto di una lista.
-

1. Sequenza crescente

Testo

Scrivere un programma che legge numeri interi dall'input finché non incontra `0`. Alla fine stampa se la sequenza di numeri inseriti era crescente oppure no.

Esempi:

- `3 16 35 100 0` → la sequenza è crescente
- `3 16 9 24 0` → la sequenza non è crescente

Prima idea: lista con flag booleano

Il primo approccio discusso è salvare tutti i numeri in una lista e poi scorrere la lista controllando se c'è qualche posizione in cui l'ordine si rompe. La proprietà che cerchiamo è:

la sequenza **non** è crescente se esiste un indice `i` tale che `lista[i] > lista[i + 1]`.

```

lista = []

n = int(input("Inserisci un numero: "))
while n != 0:
    lista.append(n)
    n = int(input("Inserisci un numero: "))

i = 0
trovato = False

while i < len(lista) - 1:
    if lista[i + 1] < lista[i]:
        trovato = True
        i = i + 1

if trovato:
    print("la sequenza non è crescente")
else:
    print("la sequenza è crescente")

```

Questa versione funziona, ma continua a scorrere la lista anche dopo aver già trovato la violazione.

Seconda idea: fermarsi appena si trova la violazione

Possiamo sfruttare la guardia del `while` per uscire non appena la condizione di crescita si rompe:

```

lista = []

n = int(input("Inserisci un numero: "))
while n != 0:
    lista.append(n)
    n = int(input("Inserisci un numero: "))

i = 0
while i < len(lista) - 1 and lista[i] < lista[i + 1]:
    i = i + 1

if i == len(lista) - 1:
    print("la sequenza è crescente")
else:
    print("la sequenza non è crescente")

```

La guardia combina due condizioni con `and`:

- `i < len(lista) - 1` evita di accedere a `lista[i + 1]` quando `i` è già all'ultimo indice;
- `lista[i] < lista[i + 1]` continua solo se l'ordine è rispettato.

Il ciclo si ferma per uno dei due motivi: o abbiamo controllato tutte le coppie senza trovare violazioni (`i == len(lista) - 1`), oppure ci siamo fermati a metà perché l'ordine si è rotto.

Tabella di tracciamento — caso crescente

lista = [3, 16, 35, 100]		
Codice	Stato	Output
lista = [3, 16, 35, 100]	lista=[3,16,35,100]	
i = 0	lista, i=0	
while i<len(lista)-1 and lista[i]<lista[i+1]:	True (0<3 e 3<16)	
i = i + 1	lista, i=1	
while i<len(lista)-1 and lista[i]<lista[i+1]:	True (1<3 e 16<35)	
i = i + 1	lista, i=2	
while i<len(lista)-1 and lista[i]<lista[i+1]:	True (2<3 e 35<100)	
i = i + 1	lista, i=3	
while i<len(lista)-1 and lista[i]<lista[i+1]:	False (3 non < 3)	
if i == len(lista) - 1:	True (3 == 3)	
print("la sequenza è crescente")		la sequenza è crescente

Tabella di tracciamento — caso non crescente

lista = [3, 16, 100, 35]		
--------------------------	--	--

Codice	Stato	Output
<code>lista = [3, 16, 100, 35]</code>	lista=[3,16,100,35]	
<code>i = 0</code>	lista, i=0	
<code>while i<len(lista)-1 and lista[i]<lista[i+1]:</code>	True (0<3 e 3<16)	
<code>i = i + 1</code>	lista, i=1	
<code>while i<len(lista)-1 and lista[i]<lista[i+1]:</code>	True (1<3 e 16<100)	
<code>i = i + 1</code>	lista, i=2	
<code>while i<len(lista)-1 and lista[i]<lista[i+1]:</code>	False (100 non < 35)	
<code>if i == len(lista) - 1:</code>	False (2 ≠ 3)	
<code>print("la sequenza non è crescente")</code>		la sequenza non è crescente

Soluzione finale: senza lista di appoggio

A lezione si osserva che per questo problema non è necessario salvare tutta la sequenza: basta ricordare il **numero precedente** e confrontarlo con quello appena letto.

```
precedente = int(input("Inserisci un numero: "))
n = int(input("Inserisci un numero: "))
condizione = True

while n != 0:
    if precedente > n:
        condizione = False
    precedente = n
    n = int(input("Inserisci un numero: "))

if condizione:
    print("la lista è crescente")
else:
    print("la lista non è crescente")
```

Perché funziona

- `precedente` contiene l'ultimo numero letto;
- `n` contiene il numero corrente;
- se a un certo punto `precedente > n`, la proprietà "sequenza crescente" è stata falsificata;

- da quel momento possiamo continuare a leggere fino allo `0`, ma `condizione` rimarrà `False`.

2. Rettangolo di asterischi

Testo

Scrivere una funzione **void** che prende in input due parametri `n` ed `m` e stampa sul terminale un rettangolo di asterischi di dimensione `n × m`.

Per esempio, con `n = 4` e `m = 6`:

```
*****  
*****  
*****  
*****
```

Idea discussa a lezione

Una funzione **void**:

- riceve dei parametri;
- esegue delle `print`;
- non restituisce un valore con `return`.

Per costruire un rettangolo servono due livelli di ciclo:

- un ciclo **esterno** che scorre le righe (`i` da `0` a `n - 1`);
- un ciclo **interno** che costruisce una stringa di `m` asterischi.

Il trucco fondamentale è non fare una `print("*")` per ogni singolo carattere, ma costruire prima la stringa intera e stamparla una volta sola alla fine di ogni riga.

Soluzione

```
def rettangolo(righe, colonne):  
    i = 0  
    while i < righe:  
        riga = ""  
        j = 0  
        while j < colonne:  
            riga = riga + "*"   
            j = j + 1  
        print(riga)  
        i = i + 1  
  
rettangolo(4, 6)
```

Cosa tiene in piedi la soluzione

Variabile	Ruolo
<code>i</code>	indice della riga corrente (ciclo esterno)
<code>j</code>	indice del carattere corrente (ciclo interno)
<code>riga</code>	stringa accumulata carattere per carattere

`riga` viene azzerata a `""` all'inizio di ogni iterazione esterna, così ogni riga riparte da capo.

3. Scacchiera di asterischi

Testo

Scrivere una funzione **void** che prende in input due parametri `n` ed `m` e stampa sul terminale una scacchiera di dimensione `n × m`.

Esempio con `n = 4`, `m = 13`:

```
*-*-*-*-*-*-*  
-*-*-*-*-*-*  
*-*-*-*-*-*-*  
-*-*-*-*-*-*
```

Idea discussa a lezione

Rispetto al rettangolo, qui non basta ripetere sempre lo stesso carattere. Bisogna decidere cosa stampare nella posizione `(i, j)`:

- se `i` e `j` hanno la **stessa parità** → `*`
- altrimenti → `-`

In modo equivalente:

riga <code>i</code>	colonna <code>j</code>	carattere
pari	pari	<code>*</code>
pari	dispari	<code>-</code>
dispari	pari	<code>-</code>
dispari	dispari	<code>*</code>

Soluzione

```
def scacchiera(righe, colonne):
    i = 0
    while i < righe:
        to_print = ""
        j = 0
        if i % 2 == 0:
            while j < colonne:
                if j % 2 == 0:
                    to_print = to_print + "*"
                else:
                    to_print = to_print + "-"
                j = j + 1
        else:
            while j < colonne:
                if j % 2 == 0:
                    to_print = to_print + "-"
                else:
                    to_print = to_print + "*"
                j = j + 1
        print(to_print)
        i = i + 1

scacchiera(4, 13)
```

Cosa ri usare dall'esercizio precedente

Questo esercizio non richiede una tecnica nuova: ri usa interamente la struttura del rettangolo. La differenza è solo nel carattere scelto per ogni posizione. Il controllo sulla parità di `i` viene fatto **fuori** dal ciclo interno, così non lo ripetiamo inutilmente a ogni colonna.

4. Elementi comuni tra due liste

Testo

Scrivere un programma che legge due liste `a` e `b` di 5 elementi dall'input. Poi costruisce una terza lista che contiene solo gli elementi comuni alle liste `a` e `b`.

Esempio:

```
a = [1, 2, 3, 4, 5]
b = [5, 3, 7, 9, 10]
output = [3, 5]
```

Idea discussa a lezione

Per ogni elemento della prima lista bisogna cercare se compare anche nella seconda. Questo porta naturalmente a un **doppio ciclo**:

- il ciclo **esterno** scorre `a` ;
- il ciclo **interno** scorre `b` .

Ogni volta che `a[i] == b[j]` , aggiungiamo `a[i]` alla lista risultato.

Soluzione

```
def leggi_lista(n):
    nuova = []
    i = 0
    while i < n:
        nuova.append(int(input("Inserisci un numero: ")))
        i = i + 1
    return nuova

a = leggi_lista(5)
b = leggi_lista(5)
c = []
i = 0

while i < len(a):
    j = 0
    while j < len(b):
        if a[i] == b[j]:
            c.append(a[i])
        j = j + 1
    i = i + 1

print(c)
```

La funzione `leggi_lista` è **pura**: riceve il numero di elementi da leggere, costruisce e restituisce una nuova lista. Non modifica nulla all'esterno.

Osservazione importante

Questa soluzione è corretta rispetto all'idea discussa, ma può inserire duplicati se uno stesso valore compare più volte nelle liste. Se ad esempio `a = [3, 3, 5]` e `b = [3, 5]` , la lista `c` conterrà `3` due volte. Se volessimo evitare i duplicati, bisognerebbe aggiungere un controllo prima di fare `append` .

5. Invertire una lista

Testo

Scrivere un programma che inverte il contenuto di una lista.

Esempio:

```
[1, 2, 3, 4, 5] → [5, 4, 3, 2, 1]
```

Prima idea: lista di appoggio

L'approccio più immediato è costruire una seconda lista leggendo gli elementi della prima dalla fine verso l'inizio:

```
lista = [1, 2, 3, 4, 5]
invertita = []
i = len(lista) - 1

while i >= 0:
    invertita.append(lista[i])
    i = i - 1

print(invertita)
```

Questa soluzione è chiara, ma usa memoria extra proporzionale alla lunghezza della lista.

Soluzione senza seconda lista: inversione in-place

L'idea migliore è invertire la lista **in-place**, cioè modificare direttamente quella originale senza crearne una seconda. Il meccanismo è scambiare gli elementi simmetricamente:

```
[1, 2, 3, 4, 5, 6]
i           j       → scambia lista[0] e lista[5]
  i         j       → scambia lista[1] e lista[4]
    i   j         → scambia lista[2] e lista[3]
```

Due indici partono dalle estremità e si avvicinano al centro. Il ciclo termina quando si incontrano.

```
lista = [1, 2, 3, 4, 5, 6]
i = 0
j = len(lista) - 1

while i < j:
    x = lista[i]
    lista[i] = lista[j]
    lista[j] = x
    i = i + 1
    j = j - 1

print(lista)
```

Perché serve la variabile temporanea

Se scrivessimo direttamente:

```
lista[i] = lista[j]
lista[j] = lista[i]
```

la prima riga sovrascrive `lista[i]` prima che lo abbiamo salvato. Alla seconda riga, `lista[i]` contiene già il valore nuovo — e assegniamo di nuovo quel valore a `lista[j]`, perdendo l'originale.

La variabile temporanea `x` serve a **conservare il valore di `lista[i]`** durante lo scambio:

```
x = lista[i]      # salva il vecchio valore
lista[i] = lista[j]
lista[j] = x      # usa il valore salvato
```

Quando fermarsi

La guardia corretta è:

```
while i < j:
```

- Se `i == j` siamo al centro di una lista di lunghezza dispari: quell'elemento non ha un simmetrico da scambiare, ci fermiamo.
- Se `i > j` ci siamo già incrociati: gli elementi sono stati già tutti scambiati.

Questo funziona sia per liste di lunghezza pari sia dispari.